

“Collaborative Story/Screenplay Writing Platform with Version History”

CO2 ASSESSMENT TOOL 2

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

BACHELOR OF

ENGINEERING IN

Artificial Intelligence & Data Science

Submitted by

192424169- LITHISH KUMAR. R

Under the

Supervision of Dr. K

Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**

August 2025



SIMATS ENGINEERING

**Saveetha Institute of Medical and Technical
Sciences Chennai-602105**



DECLARATION

I, **Lithish Kumar R 192424179**, of the **Department of Artificial Intelligence and Data Science**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **“Collaborative Story/Screenplay Writing Platform with Version History”** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place:

Chennai

Date:

Signature of the Students

INDEX TABLE (TABLE OF CONTENTS)

S.No.	Architecture Design Report Sections
1	1. System Requirements Analysis (Objectives, Functional & Non-Functional, Quality Attributes)
2	2. Selection of Software Architecture (Style & Comprehensive Technical Justification)
3	3. Software Architecture Diagram (System Components, Communication & APIs)
4	4. Component Description and Interactions (Responsibilities & System Workflow)
5	5. Evaluation of Key Quality Attributes (Scalability, Security, Performance, etc.)
6	6. Justification of Architectural Decisions & Trade-Offs (Rationale & Enhancements)
7	Appendix: Evaluation Rubrics Matrix (Self-Assessment)

1. Analyze System Requirements

The target system is a Collaborative Story/Screenplay Writing Platform with Version History. It serves as a unified workspace for professional writers, directors, and editors to construct, format, and review screenplays in real-time, while preserving a robust and auditable timeline of edits.

1.1.System Objectives

- Real-time Synchronization: Provide low-latency, conflict-free document collaborative editing so that multiple co-writers can modify screenplays simultaneously without document corruption.
- Industry Standard Formatting: Automatically format dialogues, actions, scenes, and character names according to strict screenplay formatting standards (e.g., Fountain syntax or traditional layout rules) in real-time.
- Granular Version Control: Create automatic micro-snapshots and allow writers to commit named major versions (e.g., 'Draft 1', 'Producer Review'). Implement visual diff comparisons to trace additions, deletions, and structural re-ordering.
- Secure Role-based Access Control (RBAC): Guarantee that owners, co-writers, editors, script doctors, and readers have strictly enforced, audit-logged read/write/comment capabilities.
- Multi-Format Interoperability: Enable export and import capabilities for Fountain formats, Final Draft (.fdx) XMLs, PDFs, and standard Microsoft Word formats.

1.2.Functional Requirements (Influencing Architecture)

Functional requirements dictate the structural abstractions of our system. Below are key functional areas that directly dictate the architectural layout:

Functional Module	Requirement Details	Architectural Influence
Real-time Editing Engine	Users must edit the same screenplay simultaneously, view cursor positions of other active users, and experience zero conflict overlap.	Requires WebSocket servers, Redis Pub/Sub for cross-node sync, and Yjs/CRDT state structures in memory.
Git-like Version History	Users must be able to view a visual diff, create manual commits, revert to previous points in time, and fork screenplays into alternate storylines.	Requires a database capable of storing operational history logs (deltas) alongside full snapshots. Decoupled async job workers calculate diffs.
Auto-formatting Parser	The writing interface must parse typed input in real-time, mapping scenes, dialogues, transitions, parentheticals, and character names instantly.	Demands heavy client-side JavaScript/WASM computing or worker threads to ensure no thread-blocking in the UI rendering loop.
Role-Based Sharing	Allow document sharing with explicit access bounds (read-only, comment-only, edit access).	Requires centralized API gateway authorization checks, JWT validations, and database triggers for security integrity.

1.3.Non-Functional Requirements (NFRs)

NFR Category	Target Metric	Architectural Design Decision
Latency	Editing operation sync latency < 80ms for users within same continent.	Deployment of Edge WebSocket gateways, low-overhead Y-Websocket protocol, and Redis caching of active rooms.
Availability	99.95% availability for critical editing services.	Multi-region fallback cluster, Docker containers managed by Kubernetes, and highly available PostgreSQL and Mongo replica sets.
Data Durability	Zero character loss under sudden network disconnections or client system failure.	Local browser indexDB database queues operations locally, caching updates until server acknowledgement is received.
Security	End-to-end transport layer encryption (TLS 1.3), OAuth 2.0 Auth, and encrypted document storage.	Kong API Gateway terminates TLS, verifies JWT signatures, and routes requests to microservices over private VPC.

1.4.Key Quality Attributes

To provide a solid software architecture, we identify and optimize six core quality attributes:

- **Scalability:** The platform must scale to hundreds of thousands of concurrent rooms, each managing dozens of active users. The architecture isolates memory-heavy WebSocket synchronization processes from persistent databases, using stateless gateways and cache-first workflows.
- **Maintainability:** Each sub-domain (Auth, Collaboration, Outlining, Version History, Exporting) is developed as a separate microservice. This isolation guarantees independent source codebases, allowing developers to refactor the database schema or rewrite services without risking global regression.
- **Reliability:** Data integrity is absolute. The platform guarantees eventual consistency across disconnected devices using Conflict-free Replicated Data Types (CRDTs), which mathematical formulations prove will merge cleanly once online without manual server intervention.
- **Availability:** Utilizes redundant Docker container nodes globally, automated service discovery, and database failover strategies to guarantee continuous operation even if a database primary node fails.
- **Performance:** Writers expect immediate feedback. Editing visual latency is optimized by local DOM manipulation while synchronizing changes asynchronously. Diffs and exports are run as background worker tasks to protect UI thread response times.
- **Security:** Requires deep transport layer protections (WSS, HTTPS), parameterized database interfaces, role-based authorization scopes, and automatic snapshot snapshots for backup recovery.

2. Select a Suitable Software Architecture

We select a Hybrid Software Architecture style to satisfy the divergent constraints of real-time multi-user document synchronization, massive relational metadata querying, and intensive background file generation tasks.

The hybrid system combines three architectural patterns:

- 1. Microservices Architecture: Isolates auxiliary systems (Authentication, Outlining, Billing, and Metadata) into independent, REST-compliant microservices. This allows cost-effective vertical and horizontal scaling and ensures a fault in one service (e.g. Export Service crashing under an out-of-memory file compile) does not bring down the writing editor.**
- 2. Event-Driven Architecture (EDA): Orchestrates asynchronous tasks like PDF compilation, auto-saving, version snapshot generation, and mail notifications. High-performance message brokers (RabbitMQ/Kafka) decouple services to protect live user throughput.**
- 3. Layered Client-Server (WebSocket-based): Dictates the live editing engine. A dedicated connection layer handles persistent connections to synchronize character keystrokes, presence lists, and cursors using Yjs sync protocol state structures.**

2.1. Technical Justification for Selected Hybrid Architecture

Architectural Need	Monolith / Basic MVC Limitations	Hybrid (Microservices + EDA) Advantage
Real-time collaborative editing execution	Traditional MVC server threads block on HTTP requests, causing extreme thread overhead, socket limits, and frequent database polling bottlenecks.	WebSockets establish a single persistent connection. Redis Pub/ Sub synchronizes operational buffers across server nodes instantly, minimizing database hits.
Heavy background document compilations	Exporting a 150-page screenplay to PDF or FDX format blocks CPU loops on the server, slowing down screenplays loading for other active writers.	Exports are queued into RabbitMQ. Workers process the rendering asynchronously, and a callback updates S3 Storage and alerts the writer via WebSocket.
Frequent version snapshot diff calculations	Computing lines diff on every user autosave blocks document databases and causes application lags.	A dedicated Version History service receives snapshots asynchronously, runs visual delta algorithms in isolated threads, and writes changes incrementally.
High durability under scaling pressure	A memory leak in the collaboration node crash can compromise the auth, dashboard, and files, causing global service interruption.	Each microservice is scaled independently. A failure in the Dashboard Service does not interfere with currently connected WebSocket writers.

3. Draw the Software Architecture Diagram

The diagram below represents the software architecture. It illustrates client-side editor bindings, API routing, microservices decomposition, message queues, and target databases that collaborate to manage the screenplay writing workflow.

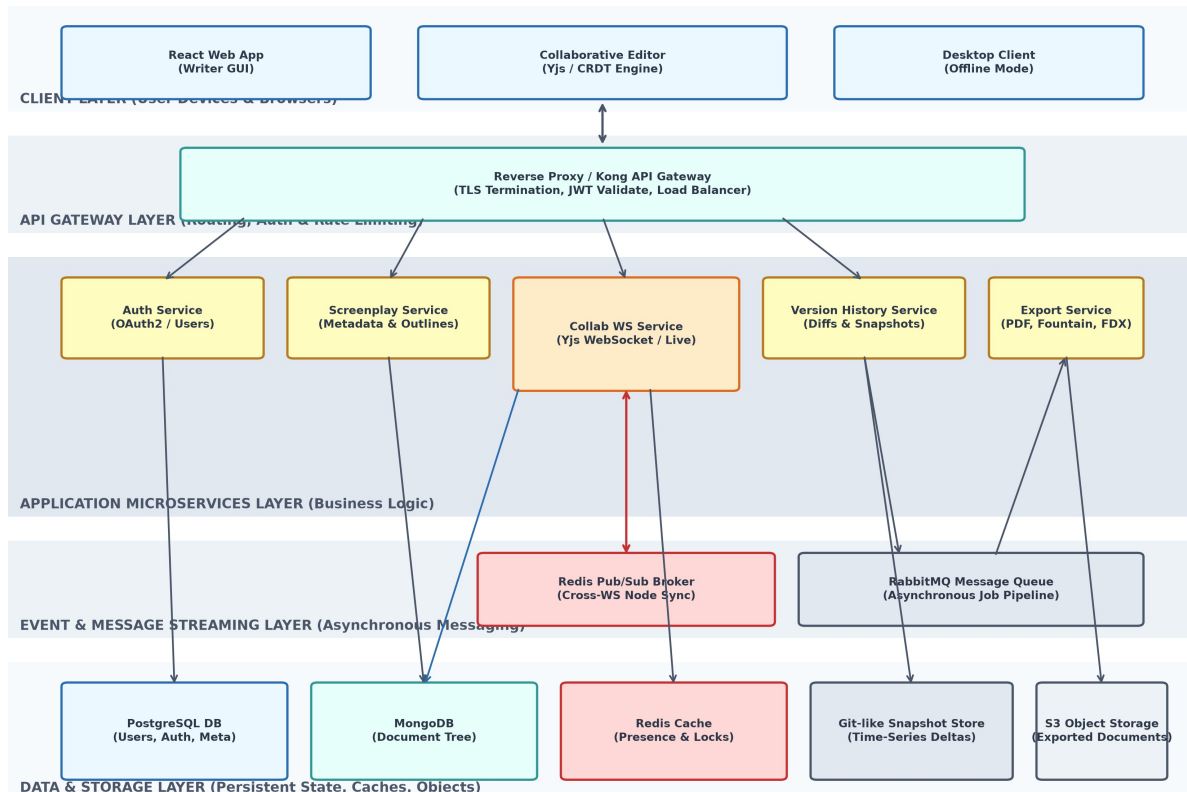


Figure 3.1: Complete Software Architecture Blueprint for the Collaborative Writing Platform

The architecture decomposes the platform into distinct, independent tiers. Below, we describe the integration paths:

- Clients establish persistent WebSocket (WSS) connections directly to the Collaboration WebSocket Service via the API Gateway, enabling instant real-time synchronization. Standard metadata modifications, outlines, and dashboard configurations use REST HTTP APIs.
- The API Gateway manages authentication via JWT tokens, handles SSL/TLS termination, and conducts rate-limiting checks to prevent Denial of Service (DoS) attacks.
- Redis cache layers store active WebSocket room details, cursor positions, and lock sequences. Redis Pub/Sub acts as a lightweight event broker to broadcast synchronization states across horizontal WebSocket nodes.
- RabbitMQ facilitates long-running asynchronous execution for tasks like screenplay exports and calculating git-like version diffs. This keeps the user interface responsive and prevents server load spikes.

- Persistent storage is split into MongoDB (ideal for document-tree structures and JSON-based screenplays), PostgreSQL (ideal for ACID transactions, user management, and authorization controls), and AWS S3 for long-term binary storage.

4. Explain Components and Their Interactions

A comprehensive description of individual system components and their communications governs the architectural soundness of this platform.

4.1.Component Roles and Responsibilities

Component Name	Core Responsibilities	Technologies
React Client & CRDT	Captures keyboard input, converts changes to conflict-free replicated data type operations, handles offline states, and synchronizes cursors.	React SPA, Yjs, Web-Workers, IndexedDB
Kong API Gateway	Validates JWT tokens, checks user session states, runs rate limit filters, and forwards client connections to internal microservice channels.	Kong, Lua, Nginx
Auth Service	Manages user registrations, credentials security, logins, multi-factor tokens, and issues cryptographic JWT session keys.	Node.js, Express, bcrypt, JWT
Collaboration WS Service	Maintains thousands of active client WebSocket rooms, broadcasts updates to connected writers, and manages conflict resolution.	Node.js (WS), Redis, Yjs-Websocket
Version History Service	Calculates character-by-character visual diffs, creates automated checkpoints, and records branch timelines for screenplays.	Python, Git-diff-match-patch, MongoDB
Export Engine Service	Assembles screenplay formats, renders PDF output using custom page templates, and exports to Final Draft (.fdx) formats.	Go (Golang), WeasyPrint, AWS S3 SDK

4.2.Component Interactions and Data Flow

Communication protocols are optimized for each channel's requirements:

- HTTP/REST with JSON: Used for CRUD operations such as logging in, updating user profiles, managing document details, and retrieving version lists. This ensures standard, simple client-server messaging for non-realtime interactions.
- WebSocket (WSS): Used for live screenplay editing. A single socket connects the client to the closest active WebSocket node, transferring binary/JSON delta packets formatted by the CRDT engine.
- AMQP (RabbitMQ): Handles background work. When a user requests an export, the frontend publishes an event. The system queues the job, runs it in the background, writes to S3, and sends a completed notification.

- gRPC / Internal REST: Used for internal microservice communication (e.g. Collaboration Service validating user access via Auth Service), using short-lived TCP paths to minimize latency.

4.3. System Workflows & Sequence Flow

The sequence below illustrates real-time collaboration workflow, tracking updates from a keystroke on User A's browser through server sync and database persistence:

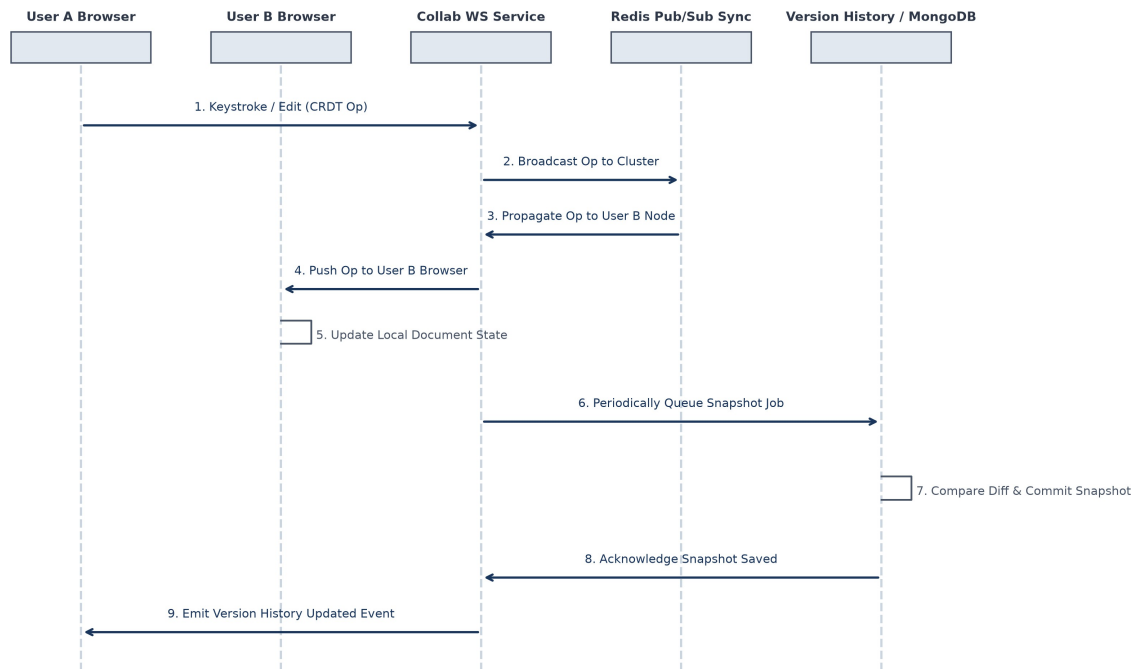


Figure 4.1: Real-time Synchronisation and Asynchronous Version History Capture Workflow

5. Discuss Quality Attributes

Evaluating how our architecture addresses critical quality metrics helps verify system viability:

Quality Attribute	Architectural Strategy	Concrete Implementation Details
Scalability	Horizontal scaling of WebSocket nodes, sharding databases, caching active room details, and isolating heavy services.	We scale node WebSocket nodes using Docker containers behind an Application Load Balancer. Redis Pub/Sub synchronizes changes across nodes. Document content tables in MongoDB are sharded by Document UUID to prevent indexing bottlenecks.
Security	Securing connections, sandboxing processes, sanitizing data inputs, and maintaining access logs.	All external connections run TLS 1.3. User tokens are validated via JWT algorithms at the Gateway. The Export microservice processes rendering runs inside isolated containers to prevent command injection, and all database operations are prepared to prevent SQL injection.
Performance	Using edge networks, local caching, client-side operations, and running heavy tasks in the background.	Static files load via CDN. The React GUI renders and caches Yjs document operations locally, keeping inputs responsive. Long-running tasks like exports or diffs are offloaded to background workers via RabbitMQ queues.
Reliability	Using conflict-free replication types, replication databases, and writing changes to log files first.	The editing engine uses CRDTs (Yjs) to resolve conflicts without central server locks. Primary PostgreSQL and MongoDB databases replicate data in real-time, providing automated failover. The collaboration service caches un-persisted states in Redis to prevent data loss.
Maintainability	Enforcing service isolation, using clean API standards, and implementing test automation pipelines.	We separate services by domain boundaries (Auth, Outlines, History). Communication channels are defined using OpenAPI specifications. Automated CI/CD pipelines run unit and integration tests before deployment, preventing regressions.

Availability	Implementing multi-region failovers, health checks, auto-healing systems, and database clustering.	Docker containers run on Kubernetes with configured health probes. If a node becomes unresponsive, Kubernetes terminates and replaces it. Database replica sets automatically promote secondary nodes if primary nodes fail, keeping services online.
--------------	--	---

6. Justify Architectural Decisions

Every architectural structure represents a balance of constraints. Below, we justify our choices, discuss the trade-offs we considered, and outline our plans for future scalability.

6.1.Rationale Behind Design Choices

Our core architectural choice is the division between real-time collaborative state management and relational metadata storage. Using CRDT (Yjs) sync engines ensures that real-time editing occurs independently of our relational database structures. This prevents database lock congestion during high-volume document edits. By offloading heavy document tasks like PDF rendering and visual diff calculations to asynchronous workers, we maintain a highly responsive user experience.

6.2.Trade-off Analysis Matrix

Decision	Selected Path	Advantages gained	Drawbacks / Mitigations
Conflict Resolution Strategy	Conflict-free Replicated Data Types (CRDT / Yjs)	Peer-to-peer merging support, native offline edits, and decentralized resolution without complex server-side locks.	Higher client-side memory usage. Mitigated by compressing old document branches and pruning active history files.
Database Selection	Polyglot Storage (MongoDB + PostgreSQL)	MongoDB easily stores document trees, while PostgreSQL handles structured metadata (users, billing, and permissions).	Higher operational cost and complexity. Mitigated by wrapping databases in standard Docker containers and orchestrating them via Kubernetes.
Real-time Connection Node Broadcast	Redis Pub/Sub Broker Sync	Provides fast, low-overhead sync across all active node processes, supporting thousands of concurrent users.	Redis is an in-memory database, so sudden restarts can clear connection caches. Mitigated by keeping persistent room lists in PostgreSQL.

6.3.Future System Integration & Long-term Maintainability

Our chosen architecture supports future enhancements through its modular API design. The API gateway separates external web interfaces from internal services, allowing us to implement features like AI-assisted story generation or automatic voice-over rendering by adding new microservices without modifying the core writing engine. All service contracts use OpenAPI specifications, ensuring that teams can work independently and develop components in different programming languages as the platform grows.

Appendix: Architecture Design Assignment Rubric Matrix

This section includes the assessment rubrics for evaluation reference:

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram & Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction & Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some	Component interactions and quality attributes are poorly explained or

	thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.		quality attributes.	missing.
Architectural Justification & Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.